🚀 005 Week 4 — English Worksheet (Intermediate)

Topic: Petting Zoo — Helper Functions · **Course:** Theme Park Creator · **Level:** Intermediate · **Time:** about 45 minutes

Your park has stalls and amenities. Now it needs a **petting zoo** — a fence around it, six cages inside, an animal in every cage, and somewhere to eat and drink.

Every cage is the same job. So you write that job **once**, then **call** it for every animal.

These are the tools you use this week:

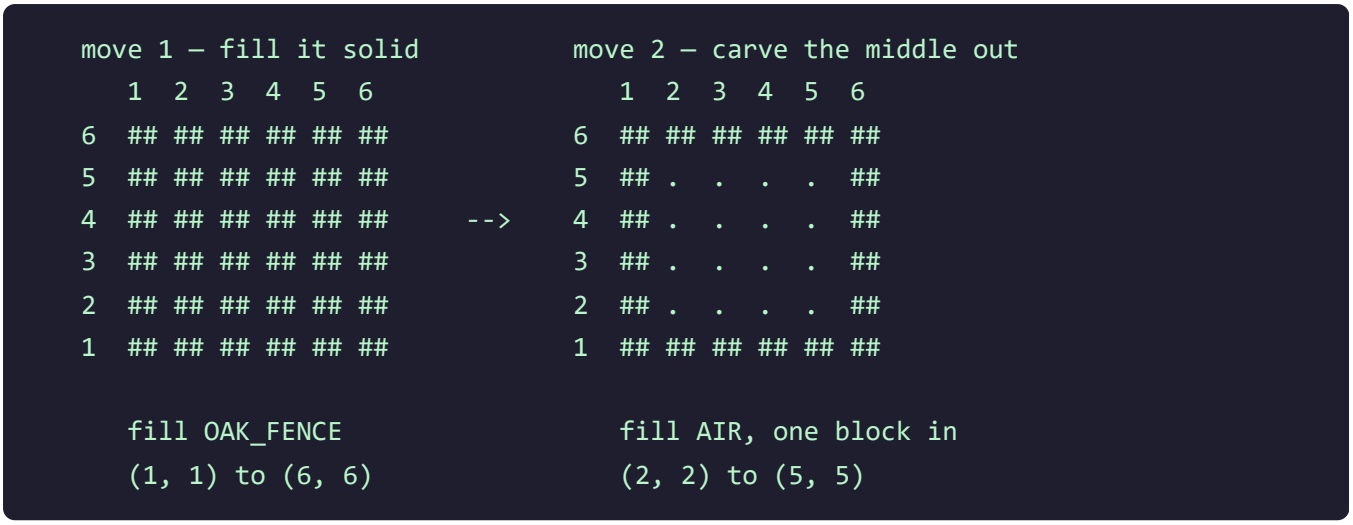
```
corner1 = pos(1, 0, 1)
corner2 = pos(10, 0, 10)
blocks.fill(OAK_FENCE, corner1, corner2, FillOperation.REPLACE)
mobs.spawn(CHICKEN, pos(6, 0, 6))

for i in range(10):
    mobs.spawn(CHICKEN, pos(6, 0, 6))
```

> `corner1` and `corner2` are the **two corners** of the box you fill. `FillOperation.REPLACE` fills the **whole solid** box. `AIR` is a block like any other — filling with `AIR` **deletes** what was there. `mobs.spawn(animal, pos(x, y, z))` puts **one** animal in **one** spot. `for i in range(10):` runs the line under it **10 times**. `OAK_FENCE` = the fence. `HAY_BLOCK` = the food. `WATER` = the drink.

These are the tools. **Turning the numbers into parameters is your job.**

1 · Solid, Then Carve 🧱

There is no "hollow box" block. So you build a box in **two moves**:



Move 2 is one block inside move 1 on every side. The edge it leaves behind is your fence.

2 · Functions 🧩

What a helper function is

	Regular function	Helper function
Who calls it?	**You** do. You type it and run it.	**Another function** does.
How big is the job?	The whole thing.	**One** small job.
In your zoo	`build_zoo`	`fence` · `feed_area`
Written differently?	No — same `def`.	No — same `def`.

> ⚠️ There is **no special keyword** for a helper. A helper is just a function that **other functions use**. The word describes its **job**, not its code.

How to tell them apart: look at who calls it. If you type it yourself, it's the regular one. If only other functions type it, it's a helper.

Functions inside functions 🪆

A function can **call** another function. When it does, it **stops and waits** — the second function runs all the way to the end, then the first one carries on from where it stopped.

Follow the arrows. `-->` means *go into*. `<--` means *finished, come back*.

```
YOU TYPE:  build_zoo(2, 2, 12, 12)
     |
     |  line 1 --> fence(OAK_FENCE, 2, 2, 12, 12)
     |                    |-- fills the outer box
     |              <-- finished. back to build_zoo.
     |
     |  line 2 --> build_cage(CHICKEN, 1, 1, 10, 10)
     |                    |
     |                    |--> fence(OAK_FENCE, 1, 1, 10, 10)
     |                    |          |-- the SAME helper, smaller
     |                    |    <-- finished. back to build_cage.
     |                    |
     |                    |-- spawn the chicken
     |                    |
     |                    |--> feed_area(1, 1)
     |                    |    <-- finished. back to build_cage.
     |              <-- finished. back to build_zoo.
     |
     |  line 3 --> build_cage(COW, 13, 1, 22, 10)
     |              <-- ... and so on
     v
```

The rule: nothing is skipped. `build_zoo` cannot get to its next line until the function it called has completely finished.

Work it out step by step:

Step 1. You type `build_zoo(2, 2, 12, 12)`. Only that.

Step 2. `build_zoo` runs line 1 → calls `fence`. `build_zoo` **stops and waits**.

Step 3. `fence` builds the outer box, ends, and hands control **back**.

Step 4. `build_zoo` runs line 2 → calls `build_cage`. `build_zoo` **stops again**.

Step 5. `build_cage` runs — and *it* calls `fence`. Now **two** functions are waiting: `build_zoo` and `build_cage`.

Step 6. `fence` ends → back to `build_cage` → it spawns, then calls `feed_area`.

Step 7. `feed_area` ends → back to `build_cage` → `build_cage` ends → back to `build_zoo`.

Step 8. `build_zoo` finally runs line 3. Repeat until all six cages exist.

So `fence` runs seven times in total — once for the zoo, once per cage — but you only ever wrote it once.

How to write a helper 🧰

Four steps, every time:

Step 1 — Find the job that repeats. The zoo fence is a box. Every cage is a box. That job runs seven times.

Step 2 — Write the job once. Give it a name that says the job.

Step 3 — Put the changing parts in the brackets. Those become **parameters**.

Step 4 — Call it, with different arguments each time.

Here is the **plan** for `build_cage`. The code is yours to write — this is only the order of the jobs:

```
def build_cage(animal, x, z, x2, z2):
    # 1. fill a solid box of fence, corner to corner
    # 2. carve the middle back to AIR, one block in
    # 3. put the animal inside
    # 4. add the feeding area
```

3 · Predict

Read each one. Write what you think happens.

```
blocks.fill(OAK_FENCE, pos(1, 0, 1), pos(10, 0, 10),
    FillOperation.REPLACE)
blocks.fill(AIR, pos(2, 0, 2), pos(9, 0, 9), FillOperation.REPLACE)
```

What is left on the ground, and how thick is it? Why those corner numbers?

```
fence(OAK_FENCE, 1, 1, 10, 10)
fence(OAK_FENCE, 13, 1, 22, 10)
```

One helper, two calls. What is different about the two boxes, and what is the same?

```
blocks.fill(WATER, pos(5, -1, 1), pos(6, -1, 2),
    FillOperation.REPLACE)
```

The `y` here is `-1`, not `0`. Where does the water go, and why is that better?

4 · Spot the Bug 🐛

Bug A — This should leave a fence ring with a space inside.

```
blocks.fill(OAK_FENCE, pos(1, 0, 1), pos(10, 0, 10),
    FillOperation.REPLACE)
blocks.fill(AIR, pos(1, 0, 1), pos(10, 0, 10),
    FillOperation.REPLACE)
```

What is left, and why?

Write the fixed lines:

Bug B — The animal should stand **inside** the cage.

```
def build_cage(animal, x, z, x2, z2):
    fence(OAK_FENCE, x, z, x2, z2)
    mobs.spawn(animal, pos(x, 0, z))
```

Where does the animal end up, and why is that wrong?

Write the fixed line:

5 · Build the Petting Zoo 🐑

In your homework world, write **one** helper and three functions that call it:

```
def fence(fence, x, z, x2, z2):
def build_cage(animal, x, z, x2, z2):
def feed_area(x, z):
def build_zoo(x, z, x2, z2):
```

`fence` builds one box: solid fill, then carve the middle to `AIR`.

`build_cage` calls `fence`, puts its animal inside, then calls `feed_area`.

`feed_area` fills `HAY_BLOCK` for food and `WATER` for drink.

`build_zoo` calls `fence` once for the outer fence, then calls `build_cage` **six** times with **six different animals**.

Only `fence` may contain a fence `blocks.fill`. Every box comes from that one helper.

Work through these in order — do not skip to step 6.

Step 1. Write `fence` first. It is the bottom of the chain, so nothing else works until it does.

Step 2. Test `fence` on its own. Call it once with numbers you can see. Fix it now.

Step 3. Write `build_cage`. Its **first** line calls `fence` — do not fill any blocks in here yourself.

Step 4. Add the animal. Middle of the box, not the corner.

Step 5. Write `feed_area`, then call it from the bottom of `build_cage`.

Step 6. Write `build_zoo`. Outer fence first, then the six cages.

Step 7. Run it. Walk in. Check every cage has an animal and food.

Animals: `CHICKEN` · `COW` · `PIG` · `SHEEP` · `RABBIT` · `LLAMA`

Cage corners that fit — front row and back row:

```
   z 13-22    [ SHEEP  ]   [ RABBIT ]   [ LLAMA  ]

   z  1-10    [ CHICKEN]   [  COW   ]   [  PIG   ]

               x 1-10       x 13-22      x 25-34
```

Write your helper and your three functions:

`fence` **is written once. Count how many times it runs when you call** `build_zoo`**. Explain your number.**

6 · Show Your Work 📸 🎥

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

3 · Your helper. Point at `fence` and say how many times it ran.

Say these out loud, filling in the blanks:

> Today I built _____.
>
> I built it using this code: _____.
>
> In this code I used _____.
>
> The hardest part was _____.
>
> That part was hard because _____.
>
> The most fun part was _____.
>
> Something new I learned was _____.

Submit ✅

Send this worksheet + your video to teacher on KakaoTalk.

Write your lines here, then say them in your video.